SISTEMAS DE AUTOMATIZACIÓN DE OFICINAS

Unidad 5: Estructuras de datos básicas

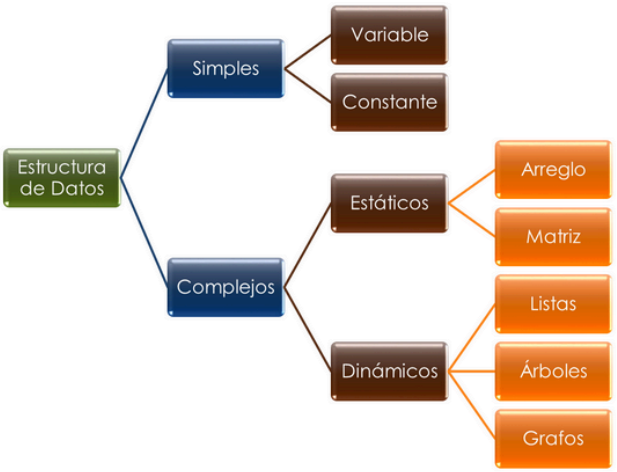
Bienvenido a la *unidad 5 de Sistemas de automatización de oficinas*, al finalizar el estudio de esta unidad tendrás un conocimiento y adquirirás habilidades de los siguientes temas:

- Arreglos. Tratamientos básicos de arreglos unidimensionales y matrices.
- Cadenas de caracteres.

Introducción

Al inicio de la materia hemos considerado a los datos como primer elemento del procesamiento de la información. Ahora introduciremos el concepto de estructura de datos antes de hablar de los arreglos.

Una estructura de datos es una forma de almacenar, representar y organizar un conjunto de datos. En el siguiente cuadro podemos observar la clasificación de las estructuras de datos.



Como se puede ver, existen desde las estructuras de datos más simples, capaces de almacenar un solo valor a la vez como las variables y las constantes. Después, dentro del grupo de estructuras de datos complejas tenemos a aquellas organizaciones que nos permiten almacenar un conjunto de datos, como las estructuras de datos estáticas y las estructuras de datos dinámicas.

Las estructuras estáticas pueden almacenar un conjunto de tamaño definido, es decir, el tamaño del conjunto no puede crecer dinámicamente en tiempo de ejecución. En cambio, las estructuras dinámicas pueden almacenar un conjunto de datos que puede crecer de forma dinámica en tiempo de ejecución. Esto quiere decir que el tamaño del conjunto cambiará según las condiciones que se den durante la ejecución del programa.

La manipulación de datos es una de las principales actividades que realizan los programas, por tal motivo se han creado varias estructuras que permiten manejar los datos de diferentes maneras, pero siempre buscando la optimización en su manejo.

Para resolver problemas que trabajan con un conjunto de variables del mismo tipo se utilizan arreglos, éstos son estructuras de datos que pueden contener datos de un mismo tipo. Los arreglos permiten manejar grandes conjuntos de datos solamente con una sola variable y mediante la utilización de índices.

En varios problemas de programación resulta necesario contar con muchas variables del mismo tipo. Se puede mencionar algunas áreas como: estadísticas, ingeniería civil, ingeniería química, entre otras.

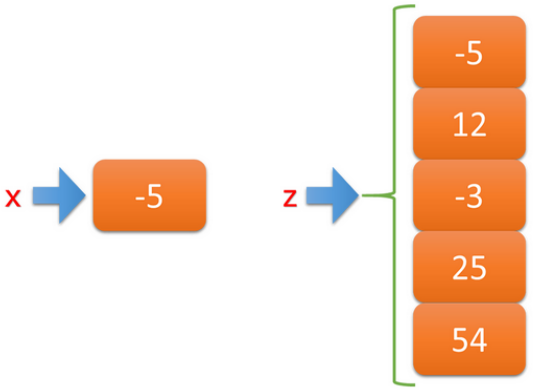
Una vez planteada la necesidad de contar con estructuras de datos, como los arreglos, es necesario pasar a estudiar cómo definir un arreglo de cualquier dimensión.

Arreglos Unidimensionales

La estructura más básica es el **arreglo**. Al arreglo también se le conoce con el nombre de *vector* o *array*.

> "Un arreglo es un grupo de posiciones de memoria contiguas, todas las cuales tienen el mismo nombre y el mismo tipo"

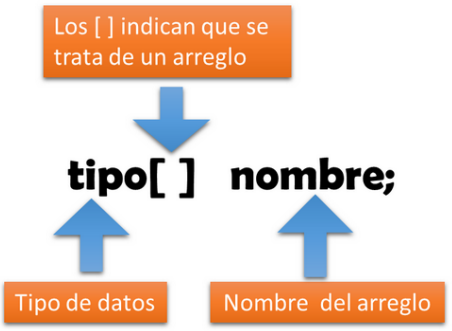
Observá en la siguiente figura la diferencia entre un arreglo y una variable estática simple. Mientras que la variable *x* sólo puede almacenar un único valor a la vez, el arreglo *z* puede contener un conjunto de *n* datos del mismo tipo. Sin embargo, tanto *x* como *z* son variables.



Declaración y creación.

Java proporciona una estructura de almacenamiento denominada array que permite almacenar muchos objetos de la misma clase e identificarlos con el mismo nombre.

Para utilizar un arreglo, como toda variable en Java, debe ser primero declarado. La forma general te la muestro en la siguiente figura:



Los corchetes nos indican que se tratará de un arreglo y no de una variable simple. En el caso de los arreglos, la declaración por sí sola no crea inmediatamente el espacio de almacenamiento, por lo que es necesario mediante el operador `new` indicar el número de casillas de memoria que utilizará el arreglo. (De ahí su carácter de estructura de datos estática) Esto implica que es necesario crear el arreglo, como se te muestra en la figura que sigue, en la cual se supone que el arreglo tuvo que haber sido declarado con anterioridad**.**



Con respecto al tamaño del arreglo:

- Debe ser un valor entero
- Puede expresarse con una constante numérica (5, 10, 2000, etc.):

```
arreglo = new int[100];
```

- También puede expresarse con una constante simbólica:

```
public static final constante = 10;
…
arreglo = new int[constante];
```

- Y también puede usarse una variable entera, siempre y cuando, ya haya sido inicializada antes de la creación del arreglo:

```
int t = 10;
arreglo = new int[t];
```

Una vez que se ha asignado tamaño de almacenamiento al arreglo se dice que ha sido "construido", y, por lo tanto, se puede emplear en el programa. Cuando esto sucede, todos los elementos en el arreglo se inicializan a cero en el caso de haber sido declarado `int, float o double; false` en el caso del tipo `boolean` o `null` en el caso de ser un arreglo de referencias.

La declaración y construcción del arreglo se puede realizar en líneas separadas o en una sola línea de código. A veces puede ser por el estilo del programador, pero también puede ser conveniente hacerlo por separado.

```
String[] respuestas = new String[10];
```

Inicialización.

Por supuesto que puedes inicializar un arreglo leyendo sus valores desde el teclado. Pero además podés inicializarlo en tiempo de edición como te muestro a continuación:

```
int[] arreglo = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

En el ejemplo se declaró un arreglo de enteros sin dimensión, la cual tomará del número de elementos contenidos entre las `{ }`. De esta forma, su tamaño es `10`, y sus elementos son:

```
arreglo[0] = 1
arreglo[1] = 2
arreglo[2] = 3
arreglo[3] = 4
arreglo[4] = 5
arreglo[5] = 6
arreglo[6] = 7
```

```
arreglo[7] = 8
arreglo[8] = 9
arreglo[9] = 10
```

De esta forma puedes inicializar arreglos de cualquier tipo de datos, como por ejemplo:

```
// Un arreglo de cadenas de texto
String[] nombres = { "Sinforoso", "Filomena", "Pancrasia", "Anacleto",
"Tiburcio", "Petra"};

// Un arreglo de caracteres
char[]  caracteres = {'#', '@', '?', '*', '/', '-', '&']};

// Un arreglo de valores booleanos.
boolean[] valores = {true, true, true, false, true, false, false, true,
false};
```

De esta manera, estás declarando, construyendo e inicializando arreglos usando sólo una instrucción.

La propiedad length

Todo arreglo declarado y construido apropiadamente es de hecho un objeto especial. Los objetos en Java tienen métodos y propiedades. En el caso de un arreglo de cualquier tipo de elementos, su única propiedad es `length`, la cual contiene el número de elementos en el arreglo.

Acceso a los elementos de un arreglo.

Una vez declarado y construido el arreglo, lo más simple que podemos hacer es acceder a los elementos de cada casilla del arreglo. Para ello, se requiere un `índice`, el cual es un valor entero que indica la posición del elemento en el arreglo.

La posición es como la dirección de un elemento en el arreglo, como una dirección de una persona que vive en un edificio de departamentos.



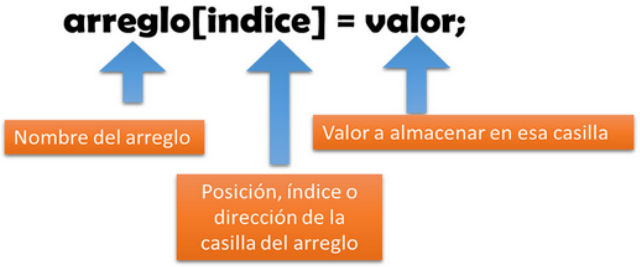
Importante destacar que la primera posición de un arreglo es cero, por lo que si un arreglo ha sido declarado y construido:

```
int[] arreglo = new int[10];
```

Tiene 10 casillas, 10 elementos, 10 espacios para almacenar datos tipo `int`; pero sólo tiene hasta la dirección 9, porque su primera dirección es 0.

Introducción de datos a un arreglo.

La forma general para introducir un dato a una casilla específica de un arreglo se muestra en la figura que sigue.



A partir de esta forma general se pueden presentar algunas variaciones como:

- El índice puede ser una constante numérica:

```
arreglo[3] = 9;
```

- El índice puede ser una variable tipo int:

```
arreglo[i] = 9;  // donde i es una variable entera e inicializada
```

- El valor puede ser un valor constante numérico que cumpla con el tipo del arreglo:

```
arreglo[2] = true // si arreglo es de tipo boolean
```

- El valor puede ser una variable igual al tipo del arreglo:

```
arreglo[3] =  x;
```

- El valor puede ser una operación cuyo resultado sea compatible con el tipo del arreglo:

```
arreglo[4] = x%y;
 //arreglo debe ser tipo int para poder almacenar este resultado
```

- El índice puede ser el resultado de una operación entera:

```
arreglo[i*2] = 12;
```

Recuperación de los elementos de un arreglo.

Se puede "recuperar" los valores almacenados en un arreglo simplemente indicando el nombre de la variable y el índice de la casilla deseada. A continuación, muestro algunos ejemplos:

```
// A la variable x se le asigna el contenido de la casilla 2 del arreglo
x = arreglo[2];

// A la variable y se le asigna el contenido de la casilla i del arreglo
y = arreglo[i];

// A la casilla 4 del arreglo1  se le asigna el contenido de la casilla 3
del arreglo2
arreglo1[4] = arreglo2[3];

// La variable x almacena la suma del contenido de la casilla 1 del
arreglo1 con el contenido de la casilla 99 del arreglo2.
 x = arreglo1[1] + arreglo2[99];

// Al contenido actual de la casilla k de arreglo se le añade 15.
arreglo[k] += 15;

// El contenido actual de la casilla 0 de arreglo se decrementa en 1.
arreglo[0]-;
```

Agilizando la manipulación de los arreglos

Cuando se requiere realizar una misma tarea sobre cada uno de los elementos del arreglo, lo más recomendable es utilizar un ciclo para recorrer cada una de sus casillas.

Uso de la estructura for para manipular arreglos

Debido a que el índice de un arreglo puede ser una variable entera que se incrementa o decrementa de 1 en 1, el ciclo `for` es el más adecuado para esta tarea. Por ejemplo, si necesitamos que todos los elementos de un arreglo sean inicializados a -1, el código mostrado en la figura que sigue realiza el trabajo.

Inicialización de un arreglo utilizando el ciclo for:

```
for (i=0; i<5; i++)
   arreglo[i] = -1;
```

Prueba de escritorio

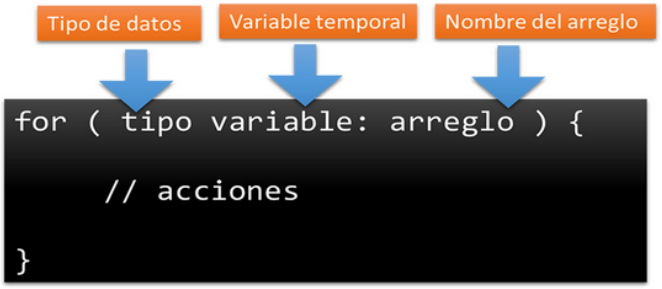
i	i<5	arreglo[i] = -1;
o	V	arreglo[0] = -1;
1	V	arreglo[1] = -1;
2	V	arreglo[2] = -1;
3	V	arreglo[3] = -1;
4	V	arreglo[4] = -1;
5	F	----

El código que muestro a continuación imprime el contenido del arreglo *x*:

```
for (int i=0; i<arreglo.length; i++)
   System.out.println(arreglo[i]);
```

Uso del for extendido.

Java introdujo desde la versión 5, una modificación a la clásica estructura de control `for`; a la que se le llama *for extendido* o *for:each*. Esta versión del `for` es especialmente útil para recorrer un arreglo. La forma general de esta estructura la puedes observar:



Simplemente expresa una forma "corta", una forma "abreviada" de utilizar el `for` para hacer un recorrido por cada uno de los elementos del arreglo. El código que se muestra en la sección anterior para imprimir cada uno de los elementos del arreglo, también puede ser escrito de la siguiente forma, utilizando el for extendido:

```
for (int i : arreglo)
   System.out.println(i);
```

Lo que hace diferente al *for* extendido del *for* clásico es que en el primero no tienes que usar al índice para acceder a un elemento específico del arreglo, sino a la variable declarada dentro del *for*.

Si observás el ejemplo, en lugar de escribir:

```
System.out.println(arreglo[i]);
```

Utilizamos:

```
System.out.println(i);
```

En donde `i` es la variable que tendrá el elemento de `arreglo[i]`.
El `for:each` es una forma más elegante de recorrer arreglos. Al no tener un contador explícito, este ciclo se puede entender como:

"Hacé esto a cada elemento en este conjunto"

En lugar de: "Hacé esto x veces"

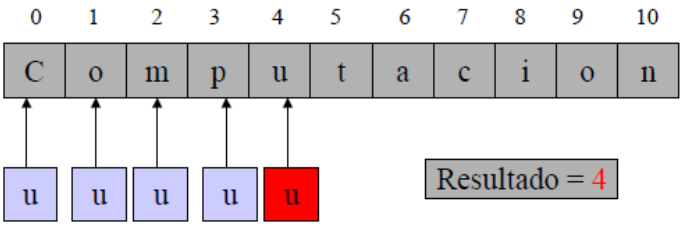
Algoritmos de búsqueda

Para encontrar un dato dentro de un arreglo, para ello existen diversos algoritmos que varían en complejidad, eficiencia, tamaño del dominio de búsqueda.
Los procesos de búsqueda involucran recorrer un arreglo completo con el fin de encontrar algo. Lo más común es buscar el menor o mayor elemento, o buscar el índice de un elemento determinado.

Búsqueda lineal o secuencial

Consiste en ir comparando el elemento que se busca con cada elemento del arreglo hasta cuando se encuentra.
Para buscar el menor o mayor elemento de un arreglo, podemos usar la estrategia, de suponer que el primero o el último es el menor (mayor), para luego ir comparando con cada uno de los elementos, e ir actualizando el menor (mayor). A esto se le llama Búsqueda Lineal.

Por ejemplo, si buscamos el elemento 'u':



Eficiencia y Complejidad

Considerando la cantidad de comparaciones:

– **Mejor Caso: O(1)** el elemento buscado está en la primera posición. Es decir, se hace una sola comparación.

– **Peor Caso: O(n)** el elemento buscado está en la última posición. Necesitando igual cantidad de comparaciones que de elementos el arreglo.

– **En Promedio**: **O(n/2)** el elemento buscado estará cerca de la mitad. Necesitando en promedio, la mitad de comparaciones que de elementos.

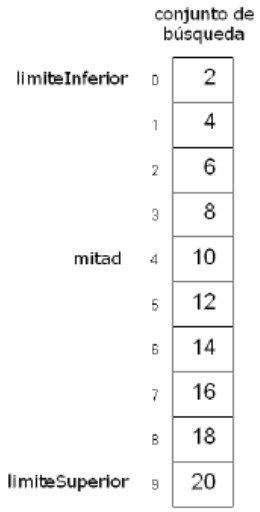
Por lo tanto, la velocidad de ejecución depende linealmente del tamaño del arreglo.

linealmente del tamaño del arreglo

Búsqueda binaria en arrays ordenados

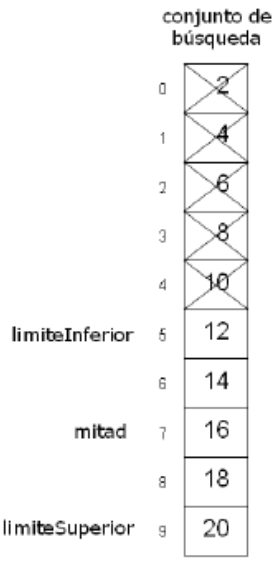
Para buscar un elemento en un array ordenado se puede aplicar la técnica de la búsqueda binaria. El conjunto de búsqueda se delimita por dos posiciones: el límite inferior y el límite superior. El algoritmo empieza la búsqueda por el elemento que está almacenado en la mitad del conjunto de búsqueda. Si el elemento almacenado en la mitad del conjunto es mayor que el valor que se busca, entonces continúa la búsqueda en la primera mitad. Si el elemento almacenado en la mitad del conjunto es menor que el valor que se busca, entonces continúa la búsqueda en la segunda mitad. Si el elemento almacenado en la mitad del conjunto es igual que el valor que se busca, finaliza el proceso. En cada comparación, el algoritmo reduce el conjunto de búsqueda a la mitad. Si durante las sucesivas reducciones del conjunto de búsqueda el límite inferior es mayor que el límite superior, entonces el valor que se busca no está en el array y finaliza el proceso.

En este ejemplo el conjunto de búsqueda tiene 10 elementos, el límite inferior coincide con el primer elemento del array y el límite superior con el último elemento del array.

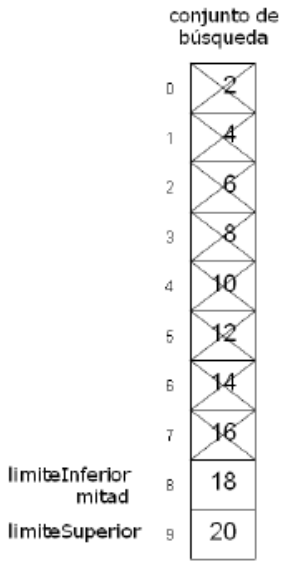


Si se aplica la búsqueda binaria para buscar el número 18, el algoritmo realiza las siguientes reducciones del conjunto de búsqueda.

Cuando se busca el número 18 en el array, en la primera iteración se compara el valor almacenado en la mitad con el 18. La mitad es la posición 4 y almacena un 10. Como 18 es mayor que 10, se descarta la primera mitad del conjunto de búsqueda y el límite inferior se hace igual a la mitad + 1. Ahora, el límite inferior es 5 y la nueva mitad es 7. Los valores del array que se han descartado en esta iteración se han tachado.



Una vez más, se compara el 18 con el valor almacenado en la mitad, que es 16. Como 18 es mayor que 16, se descarta la primera mitad del conjunto de búsqueda y el límite inferior se hace igual a la mitad + 1. Ahora, el límite inferior es 8 y la nueva mitad es 8. En la siguiente iteración se compara el valor almacenado en la posición central con el 18 y finaliza el algoritmo.



En este ejemplo, el algoritmo de búsqueda binaria ha realizado tres comparaciones para encontrar el número 18 en el array.

Durante el proceso de división del conjunto de búsqueda se modifica el valor del límite inferior o del límite superior, dependiendo de si el número que se busca está en la primera mitad o en la segunda mitad. Si durante este proceso el límite inferior es mayor que el límite superior, entonces el algoritmo finaliza porque el número que se busca no está en el array.

El siguiente programa utiliza el algoritmo de búsqueda binaria para buscar un número entre cero y 100 en un array de números ordenados.

```
public class BusquedaBinaria {

public static void main(String[] args) {
 int[] numeros = {1,2,3,4,6,7,8,9,10,15,17,20,45,51,60,68,74,75};
 int mitad;
 int limiteInferior = 0;
 int limiteSuperior = numeros.length - 1;
 int numeroBusqueda = 68;
 boolean encontrado = false;

 while ((limiteInferior <= limiteSuperior) && (!encontrado)) {
    mitad = (limiteInferior + limiteSuperior) / 2;
    if (numeros[mitad] == numeroBusqueda) {
      encontrado = true; // ¡encontrado!
    }
    else if (numeros[mitad] > numeroBusqueda) {
```

```
      limiteSuperior = mitad - 1; // buscar en la primera mitad
    } else {
      limiteInferior = mitad + 1; // buscar en la segunda mitad
    }
}

 if (encontrado)
  System.out.println("He encontrado el número");
 else
  System.out.println("No he encontrado el número");
 }
}
```

Eficiencia y Complejidad

Contando comparaciones, en el:

– **Mejor Caso: O(n/2)** el elemento buscado está en el centro. Por lo tanto, se hace una sola comparación

– **Peor Caso:** el elemento buscado está en una esquina. Necesitando **O(log2(n))** cantidad de comparaciones.

– **En Promedio:** serán algo como **O(log2(n/2))**

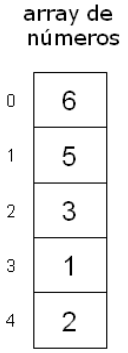
Por lo tanto, la velocidad de ejecución depende logarítmicamente del tamaño del arreglo.

Ordenamiento de arreglos

Una de las operaciones más comunes con arrays es la ordenación. Un algoritmo de ordenamiento clasifica un conjunto de datos de forma ascendente o descendente.

El algoritmo de ordenamiento "Burbuja"

El método de ordenamiento "Burbuja" (Bubble Sort) se basa en comparar cada elemento del conjunto a ordenar con el siguiente. Si estos elementos no están ordenados, entonces se intercambian. En este algoritmo es necesario revisar varias veces todo el conjunto hasta que no sea necesario realizar más intercambios.



Veamos el algoritmo para ordenar los valores almacenados en el array `numeros` utilizando el método de ordenamiento Burbuja:

```
int[] numeros = {6, 5, 3, 1, 2};
int tmp;
for (int i=0; i < numeros.length-1; i++)
  for (int j=i+1; j < numeros.length; j++)
    if (numeros[i] > numeros[j]) {
        tmp = numeros[i];
        numeros[i] = numeros[j];
        numeros[j] = tmp;
    }
```

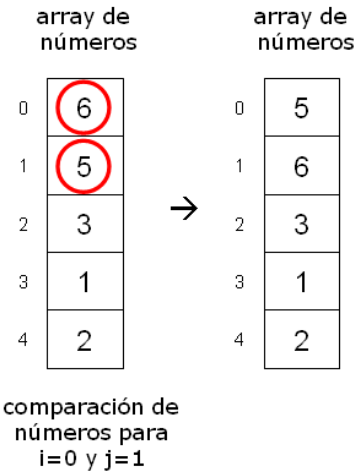
La variable `tmp` se utiliza para realizar el intercambio de los valores almacenados en las posiciones i y j del array. Primero, se almacena el valor de `numeros[i]` en `tmp`, después se almacena el valor de `numeros[j]` en `numeros[i]`, por último se almacena el valor de tmp en `numeros[j]` y finaliza el intercambio.

El algoritmo utiliza un `for` anidado. La variable de control del primer for es `i` y la del segundo `for` es `j`. La variable i del primer `for` toma los valores 0, 1, 2 y 3. Para `i = 0`, la variable `j` del segundo `for` toma los valores `1, 2, 3 y 4`. Para `i = 1`, la variable `j` toma los valores `2, 3` y `4`. Para `i = 2`, la variable `j` toma los valores 3, 4. Para `i = 3`, la variable `j` toma el valor de `4`.

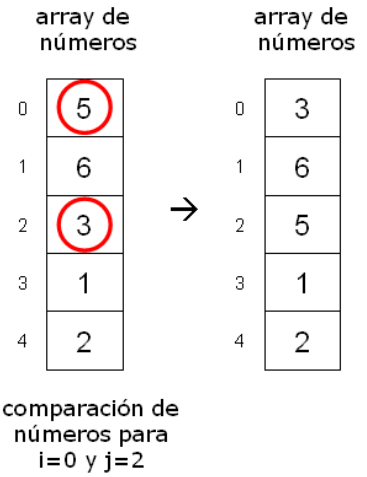
En diagramas se muestra el array de números antes y después de hacer el intercambio de los valores que se comparan cada vez.

1ra pasada:

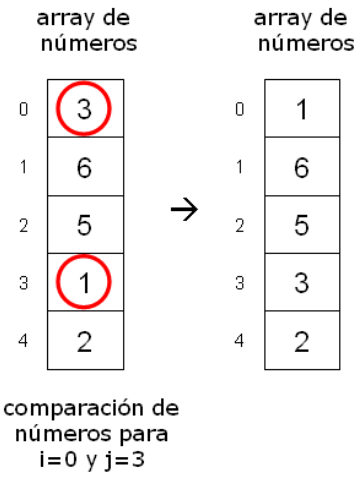
Comparación de `numeros[0]` y numeros`[1]`.



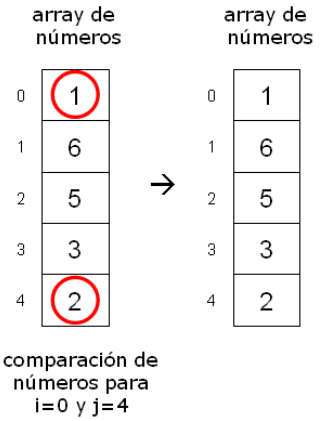
Comparación de `numeros[0]` y `numeros[2]`.



Comparación de `numeros[0]` y `numeros[3]`.

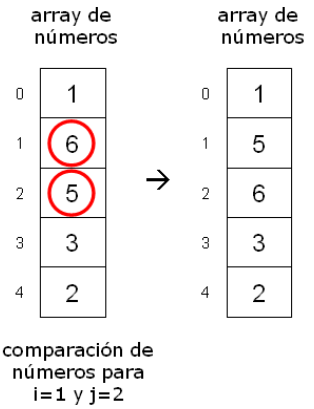


Comparación de `numeros[0]` y `numeros[4]`.

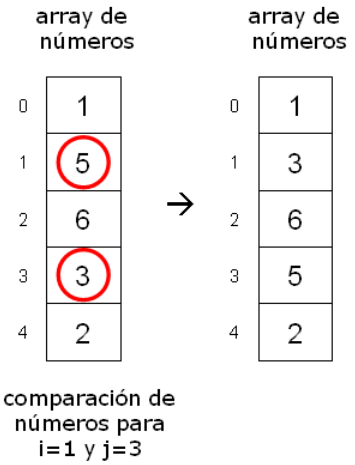


2da pasada:

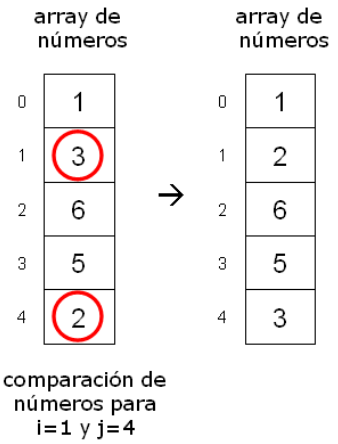
Comparación de `numeros[1]` y `numeros[2]`.


array de números
0
1
1
6
2
5
3
3
4
2
array de números
0
1
1
5
2
6
3
3
4
2
comparación de números para i=1 y j=2

Comparación de `numeros[1]` y `numeros[3]`.

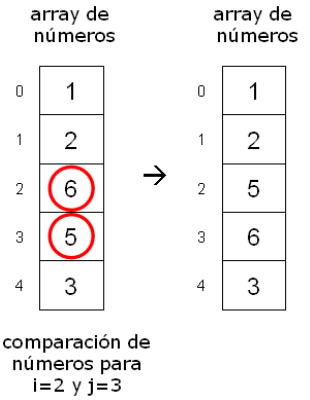


Comparación de `numeros[1]` y `numeros[4]`.

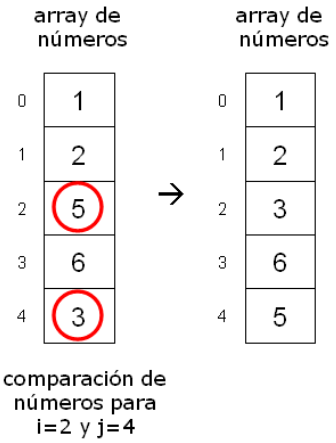


3ra. pasada:

Comparación de `numeros[2]` y `numeros[3]`.

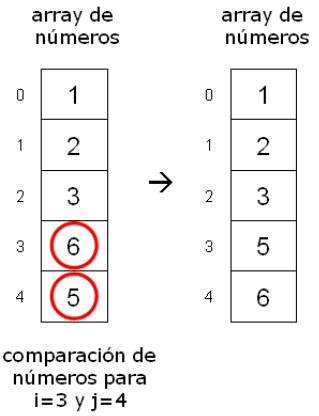


Comparación de `numeros[2]` y `numeros[4]`.



4ta. pasada:

Comparación de `numeros[3]` y `numeros[4]`.



Al finalizar el algoritmo, el array ordenado es {1, 2, 3, 5, 6}

Existen otros algoritmos de ordenamiento como el de *Selección*, el de *Inserción* y *Quicksort*. Algunos son más eficientes, mientras otros lo son menos pero más simples de entender su estrategia.

Te comparto un video que visualiza y compara 9 métodos de ordenación de arreglos:



Implementación de listas, pilas y colas en arreglos

Las listas, pilas y colas se pueden implementar a través de estructuras estáticas. Las operaciones a implementar son:

1. inserción/eliminación al principio de la lista.
2. inserción/eliminación al final de la lista.
3. inserción/eliminación en una lista ordenada.
4. inserción/eliminación en una determinada posición de la lista.

Nota: tener en cuenta las limitaciones de espacio de la estructura; en dicho caso informar tal situación y no efectuar el cambio.

Para su implementación vamos a necesitar de un índice especial, al que llamaremos Tope y que nos marcará el fin de la lista.

Esta variable Tope tendrá como valores posibles los del rango: 0 a N-1; donde N es el tamaño físico del arreglo.

Tope se declara como variable de clase para poder ser visible y accesible desde cualquiera de las funciones a implementar para modificar el tamaño lógico de la lista.

```
public class Arreglos {
	static int tope=0;
```

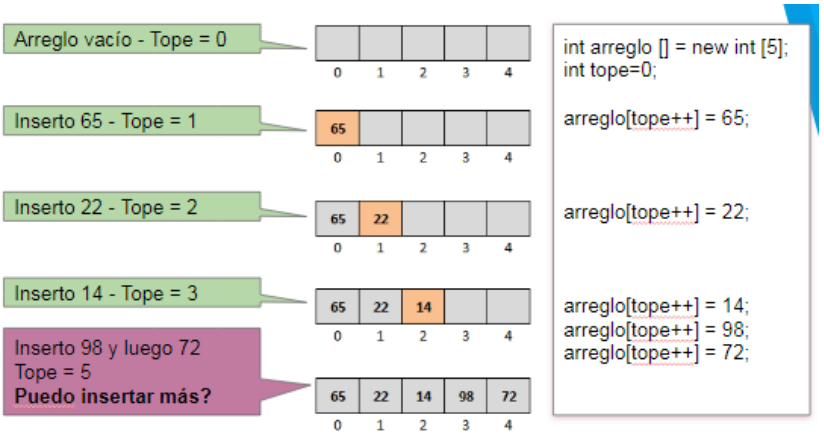
Tope indica la posición actual libre a ser ocupada al momento de la inserción. Inicialmente tendrá el valor cero.

Inserción al final (tope) de la lista

Insertar al final de la lista implica en primer lugar verificar que el arreglo no esté lleno, es decir que llegue al máximo de capacidad física reservado (Tope igual a N).

Si existe espacio disponible, se asigna el valor en la posición actual (Tope) y se actualiza Tope a la siguiente posición (Tope +1),

A continuación se muestra un esquema ejemplo:




```
public static boolean insertar_tope(int arreglo[], int valor) {
     int tam=arreglo.length;
     boolean rta=true;
     if(tam>tope)
           arreglo[tope++]=valor;
     else
           rta=false;
     return rta; }
```

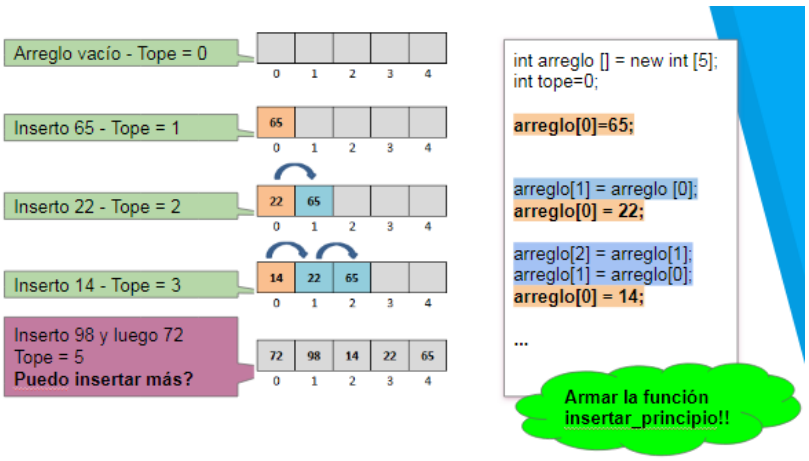
Inserción al inicio de la lista

Insertar al principio de la lista implica en primer lugar, al igual que en la estrategia de inserción al final, verificar que el arreglo no esté lleno, es decir que llegue al máximo de capacidad física reservado (Tope igual a N).

Si existe espacio disponible, en primer lugar se realiza un corrimiento de los elementos existentes una posición a la derecha, actualizando Tope.

Finalmente, se asigna el valor a insertar en la primera posición del arreglo.

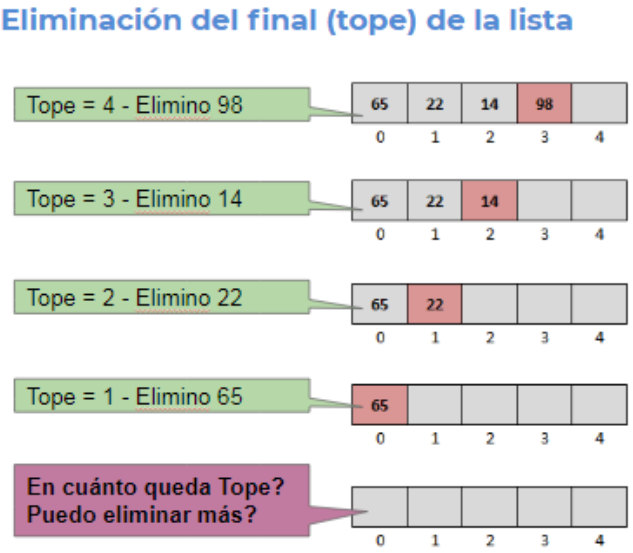
A continuación se muestra un esquema ejemplo:




```
public static boolean insertar_principio(int arreglo[], int valor) {
    int tam=arreglo.length;
    boolean rta=true;
    if(tam>tope) {
           for(int i=tope;i>0;i--)
           arreglo[i]=arreglo[i-1];
           arreglo[0]=valor;
           tope++;
    }
    else
          rta=false;
    return rta;
    }
```

Eliminación al final de la lista

Previo a la eliminación, tenemos que verificar que la lista no esté vacía. Luego actualizamos el tope lógico a la izquierda.
No es requerido limpiar o setear a cero el valor a eliminar porque Tope marca la posición actual libre, con lo cual nunca será accedida salvo para insertar el próximo valor.



A continuación se muestra un esquema ejemplo:

```
public static boolean eliminar_tope(int arreglo[]) {
 boolean rta=true;
 if(tope >0)
    tope--;
 else
     rta=false;
return rta;
}
```

Eliminación al inicio de la lista

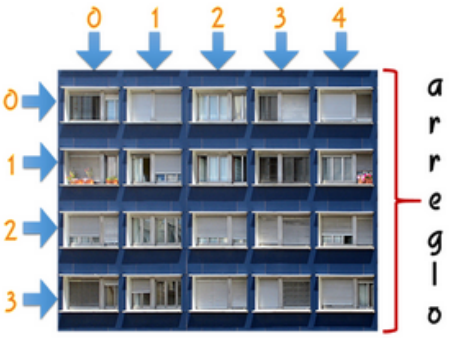
Previo a la eliminación, tenemos que verificar que la lista no esté vacía. Luego hacemos un corrimiento de todos los elementos a la izquierda y actualizamos el tope lógico también una posición a la izquierda.
No es requerido limpiar o setear a cero el valor a eliminar porque Tope marca la posición actual libre, con lo cual nunca será accedida salvo para insertar el próximo valor.

A continuación se muestra un esquema ejemplo:

```
public static boolean eliminar_ppio(int arreglo[]) {
     boolean rta=true;
     if(tope >0){
            for(int i=0; i<tope; i++)
            arreglo[i]=arreglo[i+1];
            tope--;
     }
     else
            rta=false;
     return rta;
     }
```

Arreglos Bidimensionales

Un array en Java puede tener más de una dimensión. El caso más general son los arrays bidimensionales también llamados *matrices* o *tablas*.



La dimensión de un array la determina el número de índices necesarios para acceder a sus elementos.

Los vectores que hemos visto hasta ahora son arrays unidimensionales porque solo utilizan un índice para acceder a cada elemento.

Una matriz necesita dos índices para acceder a sus elementos. Gráficamente podemos representar una matriz como una tabla de n filas y m columnas cuyos elementos son todos del mismo tipo.

La siguiente figura representa un array M de 3 filas y 5 columnas:

	0	1	2	3	4
0	1	2	3	4	5
1	2	4	6	8	10
2	3	6	9	12	15

Para acceder a cada elemento de la matriz se utilizan dos índices. El primero indica la fila y el segundo la columna.

Declaración de matrices en Java

Se crean de forma similar a los arrays unidimensionales, añadiendo un índice.

Por ejemplo:

```
Tipo_de_variable[ ][ ] Nombre_del_array = new
Tipo_de_variable[dimensión1][dimensión2];
```

También podemos alternativamente usar esta declaración:

```
  Tipo_de_variable[ ][ ]  Nombre_del_array;
  Nombre_del_array = new  Tipo_de_variable[dimensión1][dimensión2];
```

El tipo de variable puede ser cualquiera de los admitidos por Java y que ya ha sido explicado. Ejemplos de declaración e inicialización con valores por defecto de arrays, usando los distintos tipos de variables Java, serían:

```
byte[][] edad = new byte[4][3];
short ][] edad = new short[4][3];
int[][] edad = new int[4][3];
long[][] edad = new long[4][3];
```

```
float[][] estatura = new float[3][2];
double[][] estatura = new double[3][2];
boolean[][] estado = new boolean[5][4];
char[][] sexo = new char[2][1];
String[][] nombre = new String[2][1];
```

Inicialización de matrices en Java

Un array es un objeto, por lo tanto, cuando se crea, a sus elementos se les asigna automáticamente un valor inicial:

- 0 para arrays numéricos
- '\u0000' (carácter nulo) para arrays de caracteres
- false para arrays booleanos
- null para arrays de String y de referencias a objetos.

Podemos dar otros valores iniciales al array cuando se crea. Los valores iniciales se escriben entre llaves separados por comas. Los valores que se le asignen a cada fila aparecerán a su vez entre llaves separados por comas.

El número de valores determina el tamaño de la matriz.

Por ejemplo:

```
int [][] numeros = {{6,7,5}, {3, 8, 4}, {1,0,2}, {9,5,2}};
```

Se crea la matriz `numeros` de tipo `int`, de 4 filas y 3 columnas, y se le asignan valores iniciales.

Pero queremos también poder inicializar la matriz con valores que ingrese el usuario. Para ello tenemos que recorrerla. Una opción es recorrerla fila por fila, otra es recorrerla por columnas e i cargando sus valores:

Veamos, cómo sería el algoritmo de recorrido por filas:

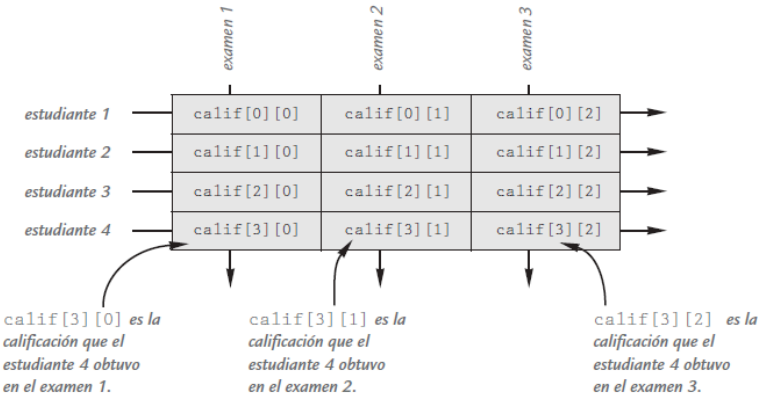
```
final int FILAS = 4, COLUMNAS = 3;
for (i = 0; i < FILAS; i++) {
            for (j = 0; j < COLUMNAS; j++) {
            System.out.print("numeros[" + i + "][" + j + "]= ");
            numeros[i][j] = e.nextInt();
            }
      }
```

Otra opción válida es utilizar en el ciclo `for` el atributo `length`:

```
for (i = 0; i < numeros.length; i++) {  //número de filas
     for (j = 0; j < numeros[i].length; j++) { // columnas de cada fila
      System.out.print("Ingrese numeros[" + i + "][" + j + "]= ");
            numeros[i][j]= e.nextInt()
     }
}
```

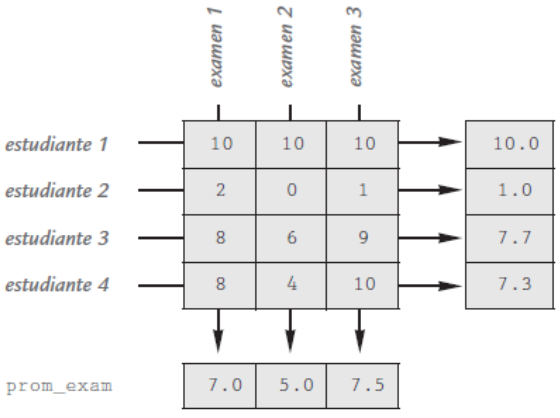
Caso de estudio

Utilizaremos un arreglo bidimensional llamado `calif` para almacenar y luego mostrar las calificaciones de un grupo de estudiantes pequeño. El grupo tiene cuatro estudiantes y sustenta tres exámenes. El primer índice del arreglo sirve para designar un estudiante, y el segundo, para designar un examen.



Puesto que los estudiantes y exámenes se numeran a partir de 1, no de 0, debemos restar uno al número de estudiantes y también al número de exámenes para obtener la variable indizada que contiene una calificación de examen específica. Por ejemplo, la calificación que obtuvo el estudiante número 4 en el examen número 1 se registra en `calif[3][0]`.

Veamos esta otra vista de `clasif`:



Podemos obtener el promedio (menor o mayor) de notas por examen, realizando un recorrido por columnas:

```
for (int num_exam = 1; num_exam <= NUM_EXAMENES; num_exam++)
{//Procesar un examen (para todos los estudiantes)
  suma = 0;
  for (int num_est = 1; num_est <= NUM_ESTUDIANTES; num_est++)
  // sumatoria de las calificaciones para el examen num_exam.
    suma = suma + calif[num_est1][num_exam1];

  System.out.print("Promedio examen "+num_exam+":" +suma/NUM_ESTUDIANTES);
}
```

Y recorriendo fila por fila podemos calcular el promedio (menor o mayor nota) de cada estudiante.

```
for (int num_est = 1; num_est <= NUM_ESTUDIANTES; num_est++)
{ //Procesar un num_est:
  suma = 0;
  for (int num_exam = 1; num_exam <= NUM_EXAMENES; num_exam++)
   //sumatoria de las calificaciones del estudiante num_est.
    suma = suma + calif[num_est1][num_exam1];
  System.out.print("promedio para el estudiante " + num_est + "es:" +
suma/NUM_EXAMENES);
```

Cadenas de caracteres

En Java y otros lenguajes orientados a objetos, los objetos son colecciones de datos relacionados que vienen con un conjunto de métodos. Estos métodos operan sobre los objetos, efectuando cómputos y, a veces, modificando los datos del objeto.

De los tipos que hemos visto hasta ahora, sólo los String son objetos. Basado en la definición de objeto, te podés llegar a preguntar "¿Cuáles son los datos contenidos en un objeto de tipo String?" y "¿Cuáles son los métodos que podemos llamar en objetos de tipo `String`?"
Los datos contenidos en un objeto `String` son las letras de la cadena.

El primer método que veremos es `charAt`, que te permite extraer letras de un `String`. Para poder almacenar el resultado, necesitamos un tipo de variable capaz de almacenar letras individuales (a diferencia de las cadenas).

Las letras individuales se llaman caracteres, y el tipo de variable que los almacena es `char`. Los chars funcionan igual que los otros tipos de datos que vimos:

```
char pepe = 'c';
if (pepe == 'c') {
System.out.println (pepe);
}
```

Los valores que simbolizan caracteres se escriben entre comillas simples (`'c'`). A diferencia de las cadenas (que se escriben con comillas dobles), los caracteres pueden contener una sola letra o símbolo. Así es cómo se utiliza el método `charAt`:

```
String fruta = "banana";
char letra = fruta.charAt(1);
System.out.println (letra);
```

La sintaxis `fruta.charAt` indica que estoy llamando el método `charAt` en el objeto llamado fruta. Estoy pasando el parámetro `1` a este método, que indica que quisiera obtener la primera letra de la cadena. El resultado es un carácter, el cual se almacena en una variable `char` llamada letra.

Cuando imprimo el valor de `letra`, me encuentro con una sorpresa:

```
a
```

a no es la primera letra de "banana". A menos que seas un científico de la computación. Por razones perversas, los científicos de la computación empiezan siempre a contar de cero. La 0ésima letra ("ceroésima") de "banana" es b. La 1ésima ("unésima") es a y la 2ésima ("dosésima") letra es n. Si querés la ceroésima letra de una cadena, tenés que pasar 0 como parámetro:

```
char letra = fruta.charAt(0);
```

Largo

El segundo método de String que veremos es length, el cual devuelve el número de caracteres de la cadena. Por ejemplo:

```
int largo = fruta.length();
```

length no toma ningún parámetro, tal como lo indica (), y devuelve un entero, en este caso 6.

Para conseguir la última letra de una cadena, podrías estar tentado de hacer algo como lo siguiente:

```
int largo = fruta.length();
char ultimo = fruta.charAt (largo); // MAL!!
```

Eso no va a funcionar. La razón es que no hay una 6ésima letra en la cadena "banana". Dado que comenzamos contando desde 0, las 6 letras se numeran del 0 al 5. Para obtener el último carácter, se tiene que restar 1 a largo.

```
int largo = fruta.length();
char ultimo = fruta.charAt (largo-1);
```

Recorrido

Algo común para hacer con una cadena es comenzar desde el principio, seleccionar cada carácter por vez, hacer algo con él, y continuar hasta el fin. Este patrón de procesamiento se llama recorrido. Una forma natural de codificar un recorrido es mediante una sentencia `while`:

```
int indice = 0;
char letra;
while (indice < fruta.length()) {
  letra = fruta.charAt (indice);
  System.out.println (letra);
  indice = indice + 1;
}
```

Este ciclo recorre la cadena e imprime cada letra en líneas separadas. Notá que la condición del ciclo es `indice <fruta.length()`, lo cual quiere decir que cuando indice es igual al largo de la cadena, la condición es falsa y el cuerpo del ciclo no se ejecuta. El último carácter al que accedemos

es aquel con índice `fruta.length()-1`.

El nombre de la variable del ciclo es `indice`. Un índice es una variable o valor usado para especificar un miembro de un conjunto ordenado (en este caso el conjunto de caracteres en la cadena). El índice indica (de aquí el nombre) a cuál querés hacer referencia. El conjunto debe ser ordenado de modo que cada carácter tenga un índice y cada índice tenga un carácter.

Como ejercicio, escribí un método que tome un `String` como parámetro y que imprima las letras al revés sin saltar de línea.

Errores en tiempo de ejecución

Los errores en tiempo de ejecución, son aquellos que no aparecen hasta que el programa ha comenzado a ejecutarse. En Java, los errores en tiempo de ejecución se llaman *excepciones*.

Hasta ahora, probablemente no has visto muchos errores en tiempo de ejecución, porque no hemos estado haciendo muchas cosas que pudieran causar uno. Bueno, ahora sí. Si usás el método charAt y le pasás un índice que es negativo, o mayor a largo-1, obtendrás una excepción: en particular, una `StringIndexOutOfBoundsException`. Probálo y fijáte cómo se muestra.

Si tu programa produce una excepción, imprime un mensaje de error indicando el tipo de excepción y en qué parte del programa se desató. Luego, el programa finaliza.

Iterando y contando

El siguiente programa cuenta la cantidad de veces que aparece la letra `'a'` en la cadena:

```
String fruta = "banana";
int largo = fruit.length();
int contador = 0;
int indice = 0;
while (indice < largo) {
  if (fruta.charAt(indice) == 'a') {
    contador = contador + 1;
  }
  indice = indice + 1;
 }
System.out.println (contador);
```

Este programa demuestra un elemento común, llamado contador. La variable contador se inicializa en 0 y luego se incrementa cada vez que encontramos una `'a'` (incrementar es aumentar en uno; su opuesto es decrementar). Cuando salimos del ciclo, contador contiene el resultado: la cantidad total de aes.

Los Strings son inmutables

Si mirás la documentación de los métodos de String, podés llegar a notar los métodos `toUpperCase` and `toLowerCase`. Estos métodos son, a menudo, una fuente de confusión, porque suena como si tuvieran el efecto de cambiar(o mutar) una cadena preexistente. En realidad, ninguno

de estos métodos, ni ningún otro puede modificar una cadena, porque la cadenas son inmutables.

Cuando llamás a `toUpperCase` en un `String`, te devuelve un nuevo `String`. Por ejemplo:

```
String nombre = "Alan Turing";
String nombreMayusculas = nombre.toUpperCase ();
```

Después de que la segunda línea se ejecuta, `nombreMayusculas` contiene el valor `"ALAN TURING"`, pero nombre aún contiene `"Alan Turing"`.

Los Strings no son comparables

A menudo, es necesario comparar cadenas para saber si es la misma, o para ver cuál viene antes en orden alfabético. Sería cómodo poder utilizar los operadores de comparación, como == y , pero no podemos.

Para poder comparar `Strings`, tenemos que usar los métodos `equals` y `compareTo`. Por ejemplo:

```
String nombre1 = "Alan Turing";
String nombre2 = "Ada Lovelace";
if (nombre1.equals (nombre2)) {
  System.out.println ("Los nombres son iguales.");
}
```

Actividad Propuesta: resolver los problemas de omegaUp unidad 5 - cadenas de caracteres.

Resumen de la Unidad 5

En esta unidad aprendimos cómo almacenar, representar y organizar los datos a través de estructuras de datos estáticas y homogéneas.

Glosario

arreglo: Colección nombrada de valores, en donde todos los valores son del mismo tipo, y cada valor es identificado por un índice.

colección: Cualquier estructura de datos que contenga un conjunto de ítems o elementos.

elemento: Uno de los valores de un arreglo. El operador [] selecciona elementos de un arreglo.

índice: Variable o valor entero usado para indicar un elemento en un arreglo.

determinístico: Programa que hace lo mismo cada vez que es llamado.

pseudoaleatorio: Secuencia de números que parecen ser aleatorios, pero que son en realidad el resultado de una computación determinística.